Zellic

Prepared for
Dev Ojha on behalf of
Valargroup

Prepared by
Jiaxin Guan
Mohit Sharma
Zellic

March 20, 2026

YPIR
Cryptographic Security Assessment

Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About YPIR 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 10

2.5. Project Timeline 10

3. Detailed Findings **10**

3.1. Malicious server can leak client secret key through selective failures (not triggered in client configuration) 11

3.2. The `multiply_add_poly` function ignores the accumulator when `crt_count` is 1 19

3.3. The `invert_poly` function and related functions produce out-of-range coefficients 21

3.4. Unchecked `u64` arithmetic in `multiply_no_reduce` can overflow 23

4. Discussion **24**

4.1. Composing PIR and selective failure attacks 25

4.2. Style comments 26

4.3. Insufficient test coverage 27

4.4. Design and threat model discussion 28

5. Assessment Results 29

5.1. Disclaimer 30

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow @zellic_io on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Valargroup from March 5th to March 19th, 2026. During this engagement, Zellic reviewed YPIR's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there completeness issues in the YPIR client?
- Can client privacy be compromised by the server?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Server-side issues
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

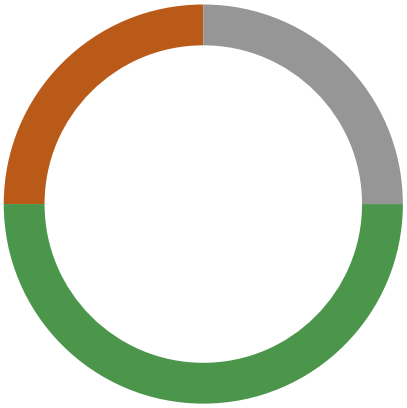
1.4. Results

During our assessment on the scoped YPIR targets, we discovered four findings. No critical issues were found. One finding was of high impact, two were of low impact, and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Valargroup in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	1
Medium	0
Low	2
Informational	1



Disclaimer

Finding 3.1. ↗ only affects a specific YPIR configuration (`simplepir=false`) that Valargroup does not use. The client's implementation relies entirely on the SimplePIR variant, which is not affected by this issue.

However, this issue has been rated as high in the report due to its potential impact on the library itself. If left unaddressed, it could affect other integrators who rely on the vulnerable configuration. Assigning a high severity helps ensure the issue is visible and not overlooked by others who may use the affected functionality.

In summary, the vulnerability is limited to an unused configuration in the client's context and does not pose a risk to their system, but it remains significant for the broader library ecosystem.

2. Introduction

2.1. About YPIR

Valargroup contributed the following description of YPIR:

> This is an implementation of the YPIR scheme for single-server private information retrieval, introduced in YPIR: High-Throughput Single-Server PIR with Silent Preprocessing ↗.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Protocol design. For novel/nonstandard protocols, we investigate the core design for potential flaws. This does not, however, apply to implementations of established protocols and algorithms. For example, when auditing a library for Yao's garbled circuits, we would not spend time on the garbling scheme itself. On the other hand, the white paper for a novel protocol would be very much considered in scope and examined thoroughly.

Applied cryptography. Zellic has a dedicated team of strong theoretical and applied cryptographers. Implementing cryptographic applications securely, like Web3 wallets, is incredibly difficult, and we help clients navigate a minefield of potential pitfalls and mistakes. We have ensured secure implementation of noncustodial wallets, ERC-4337 (AA), MPC, Shamir's secret sharing (SSS), EOAs, native multi-sig support, enclave solutions, social log-in, and key recovery, among other technologies.

Cryptographic components. We manually review the cryptography usage of the project and examine the relevant studies and standards for any inconsistencies or vulnerabilities. We ensure all of the cryptographic primitives used in the project are used and configured correctly.

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. For cryptographic applications, this includes insufficient checks on the inputs, incorrect handling of edge cases, and completeness issues in low-level arithmetic operations.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no

hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped targets itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

YPIR Targets

Type	Rust
Platform	ZCash
Target	ypir
Repository	https://github.com/valargroup/ypir ↗
Version	`b9801521301f34502496d694b2ac034857104ebc`

Programs	`client.rs`
	`lwe.rs`
	`bits.rs`
	`modulus_switch.rs`
	`utils.rs`

Target	spiral-rs
Repository	https://github.com/valargroup/spiral-rs/ ↗
Version	`f2c23c7ced192e80687b52134ee861098f764769`
Programs	`src/poly.rs`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 3.3 person-weeks. The assessment was conducted by two consultants over the course of 2.2 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
Engagement Manager
chad@zellic.io

The following consultants were engaged to conduct the assessment:

Jiaxin Guan
Engineer
jiaxin@zellic.io

Mohit Sharma
Engineer
mohit@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 5, 2026	Kick-off call
March 5, 2026	Start of primary review period
March 19, 2026	End of primary review period

3. Detailed Findings

3.1. Malicious server can leak client secret key through selective failures (not triggered in client configuration)

Target	src/client.rs		
Category	Cryptography	**Severity**	High
Likelihood	Low	**Impact**	High

Description

We present a selective-failure attack on the client-side decoding procedure of YPIR that enables full recovery of the client's RLWE secret key, in the setting where client failures can be detected. A relevant practical instance of failure detection is when a protocol sequentially composes multiple YPIR queries. The attack exploits the fact that the client decrypts server-provided ciphertexts and subsequently performs range checks on reconstructed values during decoding. By carefully crafting malicious responses, a server can cause these checks to succeed or fail depending on the value of a targeted secret-key coefficient.

This behavior yields a one-bit oracle indicating whether a decoded value crosses a protocol-defined threshold. We show how to amplify this oracle into a monotone predicate on individual secret-key coefficients by controlling the decrypted plaintext through specifically crafted ciphertexts. This enables efficient extraction of each coefficient via binary search over a small range, resulting in complete key recovery with a logarithmic number of queries per coefficient. The attack is purely passive from the client's perspective and exploits the interaction between decryption, rescaling, and postprocessing logic in the decoding routine.

Notation

We first introduce some notation to ease exposition.

We use R_q to denote the cyclotomic ring $Z[X]/ < X^d + 1 > \mod q$ for $d = 2^k$ for some k > 0. All RLWE ciphertexts will be in R_q. We will often refer to the elements of R_q as polynomials; by this we simply mean looking at their canonical embedding in $Z[X]$. Let **coeff** $: R_q \to Z_q^d$ be the mapping that views an element of R_q as a polynomial and decomposes it into its coefficients. For $p(X) \in R_q$, we use p_i to refer to the coefficient of X_i in $p(X)$ or alternatively the `i`-th entry of **coeff**(p). We use χ to denote a discrete Gaussian distribution with small width that is used to sample both the errors and challenges. The secret key will be denoted by s. We will use p to denote the plaintext modulus. Naturally, the plaintext modulus is smaller than q. We will use the *scaling factor* $\alpha =$ **round**$(\frac{q}{p})$.

The **rescale**$_{q_1,q_2}$ function is used for decoding RLWE values over R_{q_1} after canceling out the secret term. It serves two purposes:

1. Rounding out the error term
2. Descaling the plaintext, which is expected to be of the form **round**$(\frac{q_1}{q_2})\mu$ for some $\mu \in Z_{q_2}$

Roughly, when $q_1 = q, q_2 = p$, rescale roughly just computes **rescale**$_{q,p}(c) =$ **round**$(\frac{c}{\alpha})$.

The complete specification of YPIR described in the paper actually relies on two separate cyclotomic rings with different moduli and degrees, but only one of them is relevant to this attack, so we limit notation here to one ring R_q with degree d and modulus q. However, forming the failure oracle does require us to whitebox the **extract** function a bit. This involves decoding a set of RLWE-encoded ciphertexts in R_q, obtaining the corresponding plaintexts in Z_p, and then repacking these plaintexts into scalars over a different modulus. We denote this second modulus by q'.

The failure oracle

The client operates on a server response as follows:

1. In `decode_response_normal_yclient`, for a response with a single ciphertext $[a, b] \in R_q^2$ and secret key $s \in R_q$, the client computes the plaintext as `m = rescale(a*s + b, q, p)`.
2. The coefficients m_i are concatenated into a contiguous buffer using `u64s_to_contiguous_bytes`.
3. Bits from this buffer are in chunks of length `lwe_q_prime_bits`. If any single read exceeds `lwe_q_prime`, it errors out.

One thing that will be important is that the secret key is sampled from the same discrete Gaussian distribution as the error term, so that all coefficients are actually very small (in the centered representation). Looking ahead, the reason this is useful is because `s` by itself always gets scaled to 0. In other words, if you try to decrypt the vector [1, 0], the decryption will just be `s` and will scale to 0 and therefore will not cause any panic. The main idea of the attack is as follows.

To leak `s_i`, the server sends a crafted response `[a, b]` with `a = 1` and `b = b'X^{i}`. Note that s_i being close to zero (since it is sampled from a low-norm distribution) does not cause an overflow. And for a certain range offset introduced by b', an overflow, an overflow is triggered and a failure is observed. The core idea of the attack is simply to run a binary search on the offset that triggers failures and then use that to find the value of `s`. We are able to isolate our search to a single coefficient s_i at a time due to the secret being low norm and not triggering failures when no offset is added.

Impact

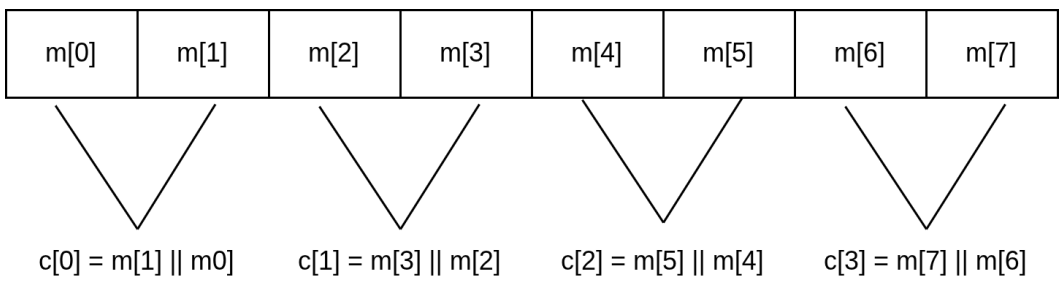
In this section we describe our end-to-end exploit and how to instantiate it for the concrete parameters relevant to the client. We instantiate a YPIR instance with the Tier 1 parameters

specified here ↗. Aside from the tooling laid out in the first section, there are two things in the concrete parameters that we can leverage. Neither of these is necessary for the theoretical idea of the attack, but will be useful in simplifying the demonstration.

1. The plaintext modulus is 2^{14}, so any 14-bit value is a valid plaintext.
2. The value `q2_bits` for the default LWE params that they are using is 28 bits, which is exactly double the plaintext length. So exactly two coefficients of the decrypted polynomial get packed into one value in `Z_{q2}`.

Using this, the attack becomes quite simple. For simplicity, assume there is only one ciphertext in the server response. (This comes without loss of generality because we can just stuff the others with [0, 0] so they are essentially NOPS. In practice, `params.rho`, which indicates the number of RLWE ciphertexts expected in a response, is 2, so we would have one real and one dummy ciphertext.)

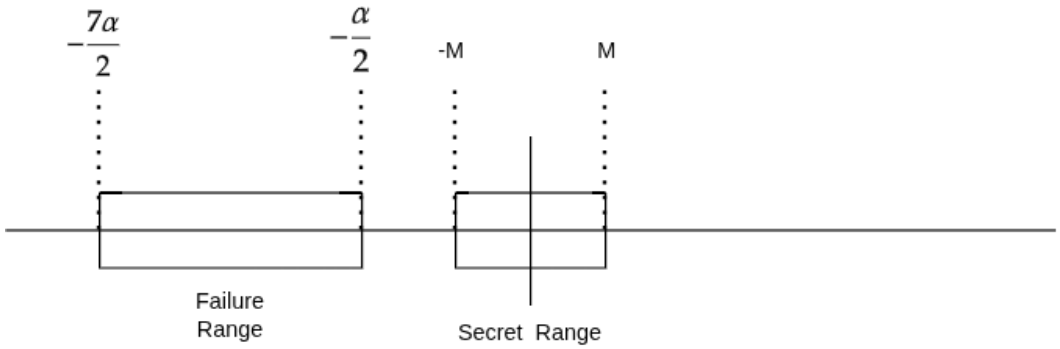
For a ciphertext $[a, b]$ let the decrypted polynomial be $m \in R_q$ and let the polynomial constructed from this after repacking be c. The failure condition is that each coefficient c fits within the new modulus q' (i.e., $c_i = m_{2i}||m_{2i+1} < q' \, \forall \, 0 \leq i < d$).



Note that the concrete value of q' in binary looks like this: `111111111110000000000000001`. That is, 12 ones followed by 15 zeros and then another 1. Since $p = 2^{14}$, we can look and the upper and lower half of it separately.

Leaking s_i for odd `i`

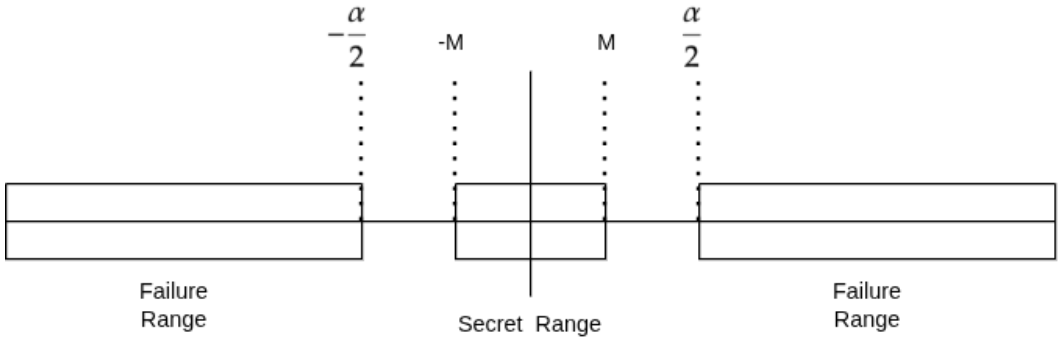
The server sends a ciphertext $[a, b]$ with $a = 1$ and $b = b'X^i$. The lower half of $c_{(i-1)/2}$ is rounded to 0 by default because it is just the small term s_{i-1}. Hence overflow must be triggered by $s_i + b'$. The upper half of q' is 11111111111100 = $2^{14} - 4$. So a failure is triggered when **rescale**$_{q,p}(s_i + b') > 2^{14} - 4$. Since **rescale**$_{q,p}(c) =$ **round**$(\frac{c}{\alpha})$, the failure condition becomes when $q - \frac{7\alpha}{2} \leq s_i + b' < q - \frac{\alpha}{2}$. In the $[-\frac{q}{2}, \frac{q}{2})$ representation, using an estimated bound (-M, M) for the secret coefficients, this can be visualized as follows:



The secret bound M for the 56-bit modulus used in the client's configuration was heuristically chosen to be 32. Therefore, the secret coefficient s_i is the lowest value s' in the range (-M, M) for which setting $b' = -\frac{\alpha}{2} - 1 - s' \mod q$ triggers a failure. Running a binary search in the given range leaks s_i in logarithmic time.

Leaking s_i for even `i`

The server sends a ciphertext $[a, b]$ with $a = 1$ and $b = b'X^i$. The lower half of $c_{(i-1)/2}$ cannot trigger an overflow by itself if the upper half is rounded to 0. Therefore, we must first set it to the threshold value $2^{14} - 4$. This can be done by either leaking the upper half first or simply adding an offset that moves the whole range (-M, M) to the failure range. The lower half of q' is just 1. Since the upper half is already at the threshold, a failure is triggered when **rescale**$_{q,p}(s_i + b') \neq 0$. This can be visualized as follows:



Therefore, the secret coefficient s_i is the highest value s' in the range (-M, M) for which setting $b' = \frac{\alpha}{2} + 1 - s' \mod q$ triggers a failure. Running a binary search in the given range leaks s_i in logarithmic time.

Below we provide a test case to execute the attack. It uses a local oracle to simulate client failures, and on a single thread setup, it manages to leak the entire secret key in under two minutes.

```
fn selective_failure_exploit() {
    let params = params_for_scenario_simplepir(
        2048,
        97792,
    );

    let q = params.modulus;
    let p = params.pt_modulus;
    let scale = q / p;
    let lwe_params = LWEParams::default();
    let lwe_q_prime_bits = lwe_params.q2_bits as usize;
    let lwe_q_prime = lwe_params.get_q_prime_2();
    let client_seed = generate_secure_random_seed();
    let mut client = Client::init(&params);
    client.generate_secret_keys_from_seed(client_seed);

    fn oracle(params: &Params, client: &Client, ct: &PolyMatrixRaw, q_prime:
    u64, q_prime_bits: usize) -> bool {
        let outer_ct = decrypt_ct_reg_measured(client, params, &ct.ntt(),
    params.poly_len).as_slice().to_vec();
        let pt_bits = (params.pt_modulus as f64).log2().floor() as usize;
        let outer_ct_t_u8 = u64s_to_contiguous_bytes(&outer_ct, pt_bits);
        let mut bits_offs = 0;
        for z in 0..1024 {
            let val = read_bits(&outer_ct_t_u8, bits_offs, q_prime_bits);
            bits_offs += q_prime_bits;
            if val >= q_prime {
                return true
            }
        }

        return false
    }

    fn oracle_query_for_guess_hb(q: u64, scale: u64, guess: u64) -> u64 {
        return (2*q - scale/2 - 1 - guess)%q;
    }

    fn oracle_query_for_guess_lb(q: u64, scale: u64, guess: u64) -> u64 {
        return  (q + (scale/2) + 1 - guess)%q;
    }

    fn threshold_value_for_hb(q: u64, scale: u64) ->u64 {
```

```
        return q - 7*(scale/2) - 100;
    }

    let mut leaked_sk = [0u64; 2048];
    let sk: &PolyMatrixRaw<'_> = client.get_sk_reg();

    let mut ct = PolyMatrixRaw::zero(&params, 2, 1);
    ct.data[0] = 1;
    let s_poly = sk.get_poly(0, 0);

    for i in 0..params.poly_len {
        if i%2 == 0 {
            let mut lo: i32 = -32;
            let mut hi: i32 = 32;

            while lo < hi {
                let mid = lo + (hi - lo + 1) / 2;
                let x = if mid > 0 {
                    mid as u64
                } else {
                    (q as i128 + mid as i128) as u64
                };
                ct = PolyMatrixRaw::zero(&params, 2, 1);
                ct.data[0] = 1;
                ct.data[params.poly_len + i] = oracle_query_for_guess_lb(q,
    scale, x);
                ct.data[params.poly_len + i + 1] = threshold_value_for_hb(q,
    scale);
                if oracle(&params, &client, &ct, lwe_q_prime,
    lwe_q_prime_bits) {
                    lo = mid;
                } else {
                    hi = mid - 1;
                }
            }

            let s: u64 = if lo >= 0 {
                lo as u64
            } else {
                (q as i128 + lo as i128) as u64
            };

            leaked_sk[i] = s;

        } else {
            let mut lo: i32 = -32;
            let mut hi: i32 = 32;
```

```
            while lo < hi {
                let mid = lo + (hi - lo) / 2;
                let x = if mid > 0 {
                    mid as u64
                } else {
                    (q as i128 + mid as i128) as u64
                };
                ct = PolyMatrixRaw::zero(&params, 2, 1);
                ct.data[0] = 1;
                ct.data[params.poly_len + i] = oracle_query_for_guess_hb(q,
    scale, x);
                if oracle(&params, &client, &ct, lwe_q_prime,
    lwe_q_prime_bits) {
                    hi = mid;
                } else {
                    lo = mid + 1;
                }
            }

            let s: u64 = if lo >= 0 {
                lo as u64
            } else {
                (q as i128 + lo as i128) as u64
            };

            leaked_sk[i] = s;
        }

        assert_eq!(leaked_sk[i], s_poly[i]);
    }

}
```

We note that the attack can be optimized further by further exploiting the low values of secret-key coefficients. We are running binary search on each coefficient separately, but one may notice that s only has small coefficients — so once enough coefficients have been leaked via binary search that the remaining coefficients form a lattice of feasibly LLL-able dimension, an attacker can start leaking (via binary search) linear combinations instead of individual coefficients. One can then use classic low-norm knapsack techniques to extract the coefficients by computing SVP in the lattice spanned by their linear combinations. This was not tested during the engagement but is expected to significantly reduce the number of queries required by a malicious server to leak the secret key as compared to individually binary searching each coefficient.

Recommendations

The presented attack can be mounted on any protocol using YPIR queries with observable failures and is therefore applicable to the YPIR library, which was in the scope of this engagement out of the box. We note that the distinguisher oracle at the core of this attack only applies to an instance of `YPIRClient` initialized with `simplepir=false`. A complete secret key leak requires multiple queries reusing the same secret key which may arise if `YClient` is used directly. We did not identify a similar library-level oracle in the SimplePIR variant of the protocol.

The audit is commissioned by Valargroup with intent of using the library in a larger project ↗, which **only** relies on the SimplePIR variant. As such their usage ↗ is **not** affected by this vulnerability.

A complete description of methodologies for resolving this class of bugs is discussed in section 4.1. ↗.

Remediation

Valargroup is not directly affected by this finding due to the absence of a library level oracle in the `simplepir=true` configuration. However, they have made `YClient` a private type in their fork and changed to sample code to no longer use it directly. The changes are implemented in PR #8 ↗ and have been merged into the valar/artifact ↗ branch. Disallowing PIR query generation directly through `YClient` prevents secret key reuse and the attack would no longer work as presented. A pull request ↗ for the same fix was also opened to the upstream repository.

3.2. The `multiply_add_poly` function ignores the accumulator when `crt_count` is 1

Target	src/poly.rs, src/arith.rs of spiral-rs		
Category	Integration Risks	**Severity**	Low
Likelihood	N/A	**Impact**	Informational

Description

The function `multiply_add_poly` in poly.rs (lines 420–427) relies on `multiply_add_modular` in arith.rs (lines 28–33) to perform a combined multiply-and-add operation. However, `multiply_add_modular` has a correctness bug when `crt_count == 1`:

```
pub fn multiply_add_modular(params: &Params, a: u64, b: u64, x: u64, c:
    usize) -> u64 {
    if params.crt_count == 1 {
        return multiply_uint_mod(a, b, params.moduli[c]);
    }
    barrett_coeff_u64(params, a * b + x, c)
}
```

When `crt_count == 1`, the early return computes only `a * b mod m` and completely ignores the accumulator `x`. The correct behavior should be `(a * b + x) mod m`, matching the behavior of the general path that calls `barrett_coeff_u64`. This causes `multiply_add_poly` to produce incorrect results whenever polynomial matrix multiplication involves an inner dimension greater than 1 (i.e., whenever accumulation across multiple terms happens).

The existing tests in spiral-rs only test full polynomial matrix multiplication on single-element (1x1) matrices, so no accumulation takes place and thus circumvents the bug. The following test, which multiplies a 1x2 matrix by a 2x1 matrix, demonstrates the issue:

```
#[test]
fn alt_full_multiply_correctness_larger_dimensions() {
    let params = get_alt_params();
    let mut m1 = PolyMatrixRaw::zero(&params, 1, 2);
    let mut m2 = PolyMatrixRaw::zero(&params, 2, 1);
    m1.get_poly_mut(0, 0)[0] = 2;
    m1.get_poly_mut(0, 1)[0] = 3;
    m2.get_poly_mut(0, 0)[0] = 5;
    m2.get_poly_mut(1, 0)[0] = 7;
    let m1_ntt = to_ntt_alloc(&m1);
```

```
    let m2_ntt = to_ntt_alloc(&m2);
    let m3_ntt = &m1_ntt * &m2_ntt;
    let m3 = from_ntt_alloc(&m3_ntt);
    // Expected: 2*5 + 3*7 = 31
    // Actual: 21
    assert_eq!(m3.get_poly(0, 0)[0], 31);
}
```

Note that the source of this finding is technically in the out-of-scope part of spiral-rs, but we flag it here because it directly affects the correctness of the in-scope `multiply_add_poly` function.

Impact

The ypir repository uses hardcoded parameters with `crt_count == 2`, so this code path is not triggered under the configuration at the time of writing. However, in future integrations, if YPIR were to be configured with different parameters where `crt_count == 1`, then all polynomial matrix multiplications involving accumulation would produce silently incorrect results. This would compromise the correctness of the entire PIR scheme.

Recommendations

We recommend modifying `multiply_add_modular` to properly include the accumulator `x` in the `crt_count == 1` path:

```
pub fn multiply_add_modular(params: &Params, a: u64, b: u64, x: u64, c:
    usize) -> u64 {
    if params.crt_count == 1 {
        return add_uint_mod(multiply_uint_mod(a, b, params.moduli[c]), x,
    params.moduli[c]);
    }
    barrett_coeff_u64(params, a * b + x, c)
}
```

Here, `add_uint_mod` should be implemented analogously to `multiply_uint_mod` by casting to `u128` before adding to prevent overflow.

Remediation

This issue has been acknowledged by Valargroup, and a fix was implemented in PR #1 ↗. This fix was merged into the avoid-avx512 ↗ branch. A pull request ↗ for the same fix was also merged into the upstream repository.

3.3. The `invert_poly` function and related functions produce out-of-range coefficients

Target	src/poly.rs of spiral-rs		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The function `invert_poly` in poly.rs (lines 462–466) negates a polynomial's coefficients modulo `params.modulus`:

```
pub fn invert_poly(params: &Params, res: &mut [u64], a: &[u64]) {
    for i in 0..params.poly_len {
        res[i] = params.modulus - a[i];
    }
}
```

When `a[i] == 0`, this computes `params.modulus - 0 = params.modulus`, which is outside the valid coefficient range `[0, modulus)`. The result should be reduced so that zero maps to zero.

A similar issue also exists at poly.rs line 486 and line 503, where operations can also produce `params.modulus` as an out-of-range coefficient when the relevant input is zero.

Impact

Coefficients equal to `modulus` instead of `0` are technically equivalent in modular arithmetic, so downstream operations that reduce their inputs may tolerate this. However, any code that assumes coefficients are strictly in `[0, modulus)` — such as comparisons, range checks, or serialization — could behave incorrectly.

The ypir repository as of the time of writing always uses the returned results after reduction, so this does not present a problem at this moment. However, producing out-of-range values is a latent correctness issue that can surface unpredictably as the codebase evolves.

Recommendations

We recommend modding the result by `params.modulus` after the subtraction or, equivalently, skipping the subtraction and using the result directly when `a[i] == 0`.

Remediation

This issue has been acknowledged by Valargroup, and a fix was implemented in PR #2 ↗. This fix was merged into the avoid-avx512 ↗ branch. A pull request ↗ for the same fix was also merged into the upstream repository.

3.4. Unchecked u64 arithmetic in multiply_no_reduce can overflow

Target	src/poly.rs of spiral-rs		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

At poly.rs line 631, the `multiply_no_reduce` function accumulates products using unchecked u64 arithmetic:

```
res_poly[z] += pol1[z] * pol2[z];
```

Both the multiplication and the addition can overflow u64. In debug builds, this will panic; in release builds, Rust silently wraps the result modulo 2^{64}. Once an overflow occurs, any subsequent reduction against the modulus q produces an incorrect result, because $(x \bmod 2^{64}) \bmod q \neq x \bmod q$ in general (since 2^{64} is not a multiple of q).

In the ypir repository, the results returned by `multiply_no_reduce` calls are eventually reduced against the modulus in packing.rs, so an overflow here would silently introduce a correctness issue.

Impact

Under the hardcoded YPIR parameters at the time of writing, this overflow does not occur. However, if the parameters are changed — for example, by increasing the modulus size or the matrix inner dimensions — the unchecked arithmetic will silently produce incorrect results. This makes it a latent correctness issue that can surface unpredictably if the parameters are changed in the future.

Recommendations

We recommend one of the following mitigations:

1. Use `checked_mul` and `checked_add` to detect overflows at runtime and surface them as explicit errors rather than silent wrapping.
2. Widen the accumulator to u128 to eliminate the overflow risk entirely for all practical parameter sizes.
3. Clearly document the parameter constraints that must hold to avoid overflow (e.g., that the inner dimension times the square of the maximum coefficient must fit in u64).

Remediation

This issue has been acknowledged by Valargroup.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Composing PIR and selective failure attacks

The standard security notion for single-server private information retrieval (PIR) is *query privacy*: the server should not be able to distinguish which index the client is querying. This is a game-based definition, and the malicious server model and the honest-but-curious one are equivalent for query privacy, since the server's input is fully determined by the client's query and the database, regardless of the server's behavior. However, an important distinction between the malicious and honest-but-curious settings is that the former does not guarantee *correctness* of the server's response. An honest-but-curious server follows the protocol faithfully, so correctness holds by construction; a malicious server, on the other hand, can return malformed responses that cause the client to either abort or produce incorrect output. In scenarios where the client does not have any further interaction with the server after the PIR query, this is mainly a usability issue, with no privacy implications. However, if the client were to have further interaction with the server dependent on the PIR output, this could compromise the query privacy guarantee.

This is what is known as a **selective failure attack** (Kushilevitz & Ostrovsky ↗, 1997; Kiraz & Schoenmakers ↗, 2006; and Huang, Katz, & Evans ↗, 2013). A malicious server crafts malformed responses in an attempt to learn the client's query by utilizing an extra bit of information that corresponds to whether the client aborts or accepts the response. In the particular setting of composing two rounds of a PIR protocol, the malicious server can exploit whether the client proceeds with a second-phase query as this extra bit; by crafting a malformed first-round response that only causes failure for certain queried indexes, the server can infer information about the client's first-round query based on whether a second-round query is observed.

This class of attack is well-known in the PIR literature. Defending against it in full generality requires a stronger primitive such as authenticated PIR or malicious-secure PIR, which guarantees both correctness and privacy against a malicious server (see Colombo et al. ↗, 2023, and Alon & Beimel ↗, 2025). One could also augment YPIR with verifiability techniques (e.g., de Castro & Lee ↗, 2024) to detect malformed responses.

However, for a two-phase composition where the client always terminates after sending exactly two queries, a simpler mitigation exists: **the client should always send a second-round query regardless of whether the first round succeeded or failed**. If the first-round response appears invalid, the client sends a dummy second-round query instead of aborting. This ensures that the server's view is computationally indistinguishable from a client that always makes two queries for a fixed index, since the query privacy property of the second round guarantees that the server cannot distinguish a dummy query from a real one. This approach is sufficient as long as the server cannot observe any additional information (such as whether the client accepted or rejected the second-round response or any public information that the client might publish on chain) beyond the two queries themselves.

We note that this mitigation is specific to the two-phase setting. If the protocol were extended to allow retries, where the client restarts the full protocol upon receiving a bogus answer, the selective failure attack would resurface, since the server could observe whether a retry occurs. In such a setting, a full malicious-secure PIR protocol would be necessary.

4.2. Style comments

In src/bits.rs, the following piece of code is unreachable:

```
let can_take = std::cmp::min(8 - bit_pos, remaining_bits);

// Omitted...

remaining_bits -= can_take;
if remaining_bits == 0 {
    break;
}

// If we did not use a whole byte and there are still remaining bits
// take the lower bits from this byte and the higher bits from the next byte
if bit_pos + can_take < 8 {
    let from_next_byte = data[i] & ((1 << (bit_pos + can_take)) - 1);
    result |= (from_next_byte as u64) << (num_bits - remaining_bits);
    remaining_bits -= bit_pos + can_take;
}
```

The `if bit_pos + can_take < 8` branch will never be executed:

1. If `8 - bit_pos < remaining_bits`, `can_take` is set to `8 - bit_pos`, so we have `bit_pos + can_take = 8`, not entering the if branch.
2. If `8 - bit_pos >= remaining_bits`, `can_take` is set to `remaining_bits`. Then, after executing `remaining_bits -= can_take`, `remaining_bits` will be 0, breaking out of the outside for loop.

In src/modulus-switch.rs, `PolyMatrixRaw::recover` contains a typo when calculating the `bit_lengths` of the arguments `q1` and `q2`. The variable names are switched, but this does not present a functional issue as their usage is inverted once more, making it correct.

```
let q_1_bits = (q_2 as f64).log2().ceil() as usize;
let q_2_bits = (q_1 as f64).log2().ceil() as usize;
```

4.3. Insufficient test coverage

The `ypir` and `spiral-rs` repositories appear to be mainly intended for proof-of-concept and benchmarking purposes and have limited testing in place. We recommend caution when using them directly in production out of the box. While the implementation is of reasonable code quality and has basic test coverage, we recommend adding extensive testing — unit, integration, and end-to-end testing — for the parts of the repositories that will be utilized. The current test suite does not fully cover security-relevant behaviors in an end-to-end instantiation of the scheme. While core functionality has some tests, several areas remain under-tested, making it difficult to confirm correctness under edge cases, failure conditions, and adversarial scenarios.

Specifically, the following areas require additional coverage:

- Behavior of the client module under malformed server responses.
- Behavior of the client module in suboptimal environments, e.g when running on a host with insufficient entropy sources leading to failures in randomness generation.
- Behavior of the scheme under a variety of different parameter sets, including non-standard ones that may allow insecure deployments.

Insufficient test coverage increases the likelihood that:

- Bugs or vulnerabilities remain undetected until production.
- Refactors or minor changes unintentionally break existing functionality.
- Critical security assumptions, such as invariant guarantees or permission checks, are violated silently.
- Tests that assert key invariants and system properties after relevant operations.

Comprehensive testing improves code correctness, auditability, and developer confidence, and reduces the risk of costly post-deployment fixes. We recommend using 4.4. ↗ as a guideline for identifying critical code-paths for extensive testing.

Note: During remediations, Valargroup implemented test coverage improvements to `ypir` and `spiral-rs` in the following pull requests:

- ypir PR #9 ↗
- ypir PR #10 ↗
- spiral-rs PR #9 ↗
- spiral-rs PR #10 ↗

4.4. Design and threat model discussion

The project is an implementation of YPIR ↗, a protocol for the PIR protocol built on RLWE-based homomorphic primitives. The client generates encrypted queries and locally decrypts and decodes server responses to recover database items. The scope of this audit focuses on the client-side cryptographic logic, particularly the response decoding pipeline, including decryption, rescaling, and bit-level reconstruction of packed values.

The server is treated as an external party that returns responses to client queries. The audit does not assume correctness or honesty of the server and instead explicitly considers adversarial behavior in response generation. Networking, transport-layer guarantees, and server-side implementation details are out of scope except insofar as they influence client-side security.

The intent of this scope is to be used as a client library for PIR in settings where the server is untrusted.

Adversary model

The adversarial model considered in this audit is as follows:

- The server is assumed to be malicious and may deviate arbitrarily from the protocol specification. In particular, the server may return arbitrary, adversarially crafted ciphertexts instead of valid protocol responses.
- The adversary has full control over all server-side computation and can adaptively choose responses based on prior interactions.
- The adversary is computationally bounded and does not break underlying hardness assumptions such as RLWE or LWE.
- The client may use sequentially composed queries; therefore, adversary may observe any externally visible side effects of client execution, including
    - success vs failure (e.g., panics, aborts), modeled by observing whether or not a second query arrives, and
    - timing differences, modeled by measuring latency between consecutive queries.
- The client is assumed to run in an isolated environment, so the adversary does not have direct memory access or local side-channel access. Only network-observable effects are considered.

Security goals

This audit was conducted with the intent of ensuring that the following security goals are met:

- **Query privacy.** The server should not learn any information about the client's queried index beyond what is revealed by the protocol's output. In particular, malicious responses should not enable leakage of the client's secret key or query-dependent information.

- **Client key confidentiality.** The client's RLWE secret key must remain hidden even under adversarially crafted responses. The server should not be able to extract any partial or full information about the secret key through interaction with the client.
- **Robustness to malformed responses.** The client should safely handle arbitrary server responses without exposing secret-dependent behavior. In particular, response decoding should not introduce selective-failure channels or other side channels that depend on secret values.
- **Complete decoding behavior.** The decoding pipeline (decryption, rescaling, and unpacking) should not introduce completeness issues that prevent the correct processing of valid responses.

5. Assessment Results

During our assessment on the scoped YPIR targets, we discovered four findings. No critical issues were found. One finding was of high impact, two were of low impact, and the remaining finding was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.